Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Chair of Aerospace Information Technology

Online Floatsat 2021
Attitude Determination – Part 2

Julian Rothe

What are we going to learn?

Part2: Problems, Quaternions, Matlib and Quaternion Filter

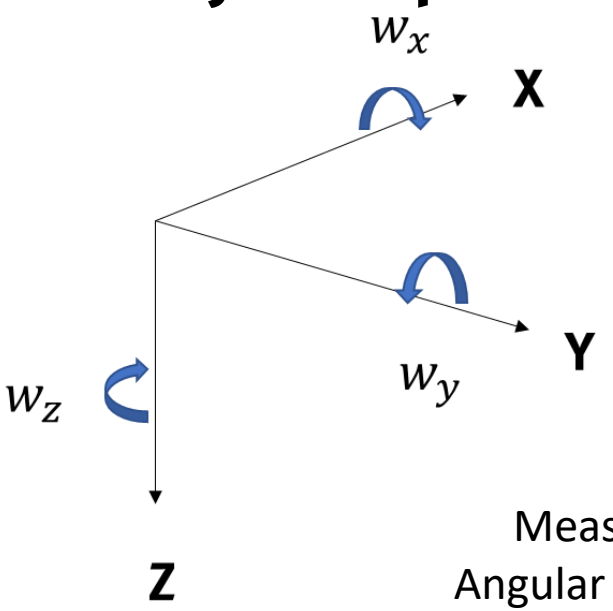
- Problems in the reality
- What are Quaternions, why do we need them?
- RODOS Matlib – This will make things much easier!
- Improve our previous complementary filter by using quaternions
- Implement an embedded quaternion attitude filter

IMU Basics

- IMU = Inertial Measurement Unit
- Consists of (one of) the following sensors:
  - Gyroscope → Angular Velocity ω
  - Accelerometer → Linear Acceleration a
  - Magnetometer → Magnetic Field m
- In the exercises we use all of the sensors above

Real world – Gyroscope

Gyroscope measures the angular velocity in 3 axis:



$$\omega = \left[\omega_x \; \omega_y \; \omega_z\right]^T$$

$$\omega = \hat{\omega} + b_g + n_g$$

Measured Angular Velocity

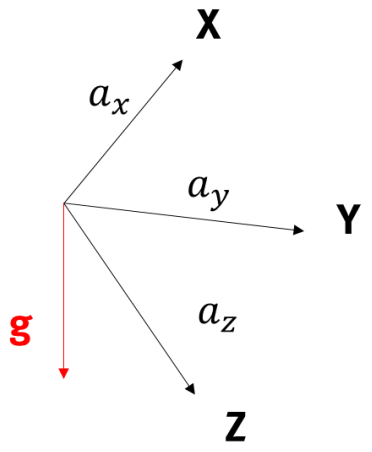
Real Angular Velocity

Bias

White Noise

Real world – Accelerometer

Accelerometer measures the linear acceleration in 3 axis



$$a = \left[a_x \; a_y \; a_z\right]^T$$

$$a = \mathrm{R}^T(\hat{a} - g) + b_a + n_a$$

Measured Linear Acceleration

Rotation to Body Frame

Real Linear Acceleration

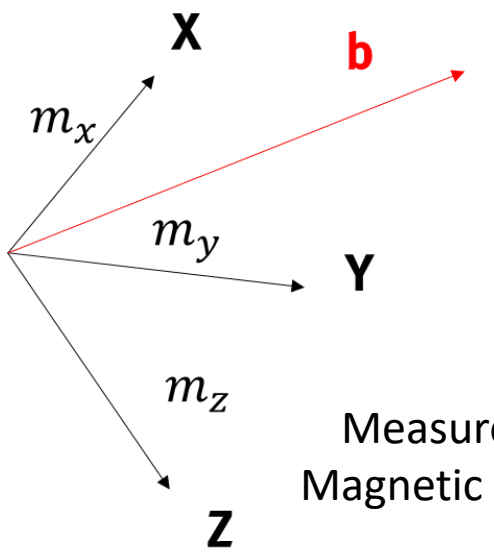
Gravity Vector

Bias

White Noise

Real world – Magnetometer

Magnetometer measures the magnetic field in 3 axis



$$m = \left[m_x \; m_y \; m_z\right]^T$$

$$m = R^T \hat{m} + b_m + b_m$$

Measured Magnetic field

Rotation to Body Frame

Real Magnetic field

Bias

White Noise

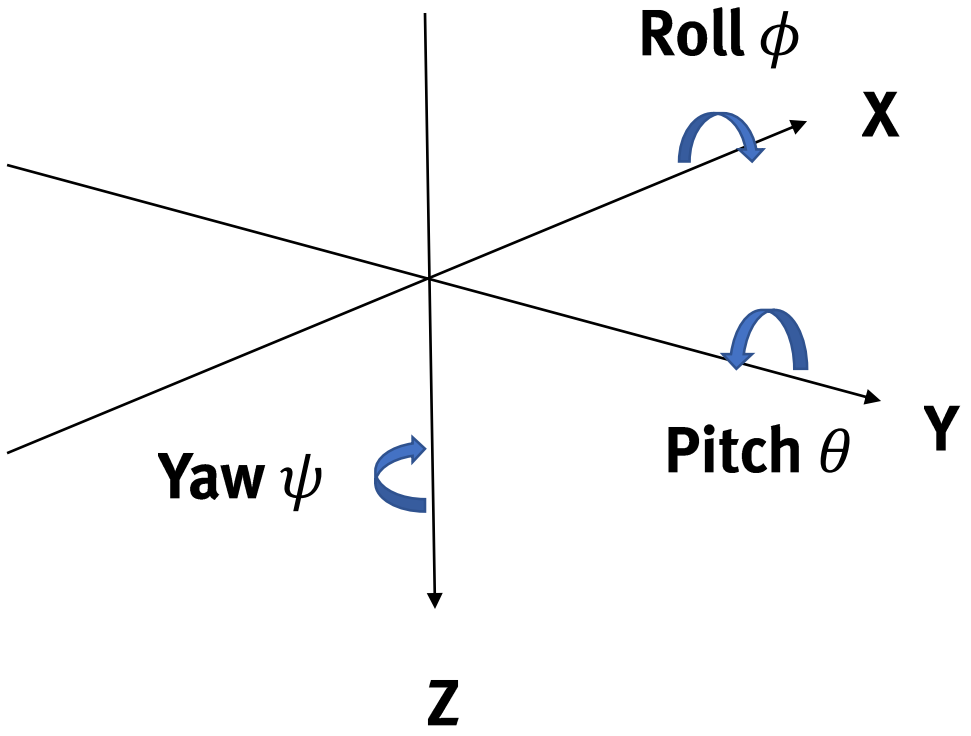
Real world – Other Problems

Other Problems in the reality include:

- **Update rates of the sensors used in the filter are different**
- **Usage of different frames**

  → Direction of movement/rotation is not the same
- **Rotating / Moving to fast for the sensor to measure**

Tait-Bryan Angles



Tait-Bryan Angles

- Tait-Bryan Angles describe the orientation by 3 consecutive rotations in a fixed order (we will use Yaw – Pitch – Roll)
- Euler Angles (X-Y-X) $\neq$ Tait-Bryan Angles (i.E. Z-Y-X)
- Rotations around each axis can be described by rotation matrices

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$

Gimbal Lock

- Gimbal Lock = Loss of one degree of freedom of rotation

  → When two rotational axis align

→ Solution: We need an additional degree of freedom

Quaternions

- **A quaternion is a hypercomplex number of rank 4**

$$q = q_0 + q_1\, i + q_2\, j + q_3\, k$$

- **We can also write**

$$q = [q_0 \;\; \boldsymbol{q}_v^T]^T = [q_0 \;\; q_1 \;\; q_2 \;\; q_3]^T$$

- **The norm of a quaternion is**

$$Norm(q) = \; \|\boldsymbol{q}\| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$$

Quaternions

- **The Hamilton-Product of two quaternions p and q**

$$\boldsymbol{p} \otimes \boldsymbol{q} = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}_v^T q_v \\ p_0 \boldsymbol{q}_v + q_0 \boldsymbol{p}_v + \boldsymbol{p}_v \times \boldsymbol{q}_v \end{bmatrix} = \begin{bmatrix} p_0 & -p_1 & -p_2 & -p_3 \\ p_1 & p_0 & -p_3 & p_2 \\ p_2 & p_3 & p_0 & -p_1 \\ p_3 & -p_2 & p_1 & p_0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

- **A vector in the navigation Frame N can be described from the body Frame B**

$$\begin{bmatrix} 0 \\ v_B \end{bmatrix} = \boldsymbol{q}_{NB} \otimes \begin{bmatrix} 0 \\ v_N \end{bmatrix} \otimes \boldsymbol{q}_{NB}^*$$

$$v_B = \boldsymbol{R}_{NB}\, \boldsymbol{v}_N$$

Quaternions

- **To display data we can convert Quaternions in Tait-Bryan Angles**

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} atan2(2(q_0 q_1 + q_2 q_3), 1 - 2(q_1^2 + q_2^2)) \\ asin(2(q_0 q_2 - q_1 q_3)) \\ atan2(2(q_0 q_3 + q_1 q_2), 1 - 2(q_2^2 + q_3^2)) \end{bmatrix}$$

- **Converting back by:**

$$\boldsymbol{q} = \begin{pmatrix} \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) - \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) - \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) \end{pmatrix}$$

RODOS Matlib – Introduction

RODOS Support Library containing a lot of functionality

- **Contains different useful datatypes**

  → i.E. Vectors, Quaternions (!), Matrix

- **Most operators and functions are implemented as well!**

  → i.E. Scalar/Cross Product, Quaternion inverse, Rotations

RODOS Matlib – Introduction

RODOS Support Library containing a lot of functionality

- **Using the matlib will make your life much easier ☺**

→ `#include "matlib.h"`

RODOS Matlib – Vectors

- Vectors of any length can easily be created and accessed
- Special datatype for the most used vector of length 3
- Most operators and functions are implemented and easy to use

```
Vector_<12> vector;

vector.r[0][0] = 1.0;
vector.r[0][1] = 2.0;
vector.r[0][2] = 3.0;
```

```
Vector3D vector3;

vector3.x = 1.0;
vector3.y = 1.0;
vector3.z = 1.0;
```

```
Vector3D a,b,c, axb, c_n;

c = a + b;
axb = a.cross(b);
c_n = c.normalize();
```

RODOS Matlib – YPR

- Special Vector datatype: YPR (Yaw Pitch Roll)
- Similar to Vector3D with formatting to represent angles

```
YPR ypr;

ypr.roll = 0.0;
ypr.pitch = 0.0;
ypr.yaw = 0.0;
```

RODOS Matlib – Quaternions

- Special Vector datatype: Quaternions
- Consists of a float (q0 or qw) and a Vector3D (x,y,z)
- Most functions and operators are implemented

```
Quaternion quat, q,p, q_conj;

quat.q.x = 0.0;
quat.q.y = 0.0;
quat.q.z = 0.0;
quat.q0 = 1.0;

quat = q*p;
q_conj = -q;
q = q.normalize();
```

RODOS Matlib – Matrix

- Matrices of any length can easily be created and accessed
- Special datatype for the most used matrix of length 3 by 3
- Most operators and functions are implemented and easy to use

```
Matrix_<10, 3> matrix;

matrix.r[0][0] = 1.0;


Matrix3D RotMat;

RotMat.r[0][0] = 1;
RotMat.r[1][1] = 1;
RotMat.r[2][2] = 1;
```

RODOS Matlib – Important Functions

- **Quaternion ←→ YPR or Vector?**

```
quat = ypr.toQuaternion();
quat = Quaternion(ypr);
 ypr = quat.toYPR();
```

- **Rotate a Vector using a quaternion or ypr**

$$\begin{bmatrix} 0 \\ v_B \end{bmatrix} = \boldsymbol{q}_{NB} \otimes \begin{bmatrix} 0 \\ v_N \end{bmatrix} \otimes \boldsymbol{q}_{NB}^{*} \quad \rightarrow$$

```
b = a.qRotate(quat);
c = a.yprRotate(ypr);
```

Task2.1: Matlib

Look at the different datatypes and functionalities of the matlib

- **Implement vectors, matrices, quaternions and use them**
- **Code is given for you to experiment with the matlib**
- **Solve the easy tasks given in the code (and do more)**

→ This will make the following exercises much easier

Goal

- Determine the Attitude (= orientation) of a system
- The Attitude is fixed to a given Frame (i.E. world frame)
- Can be given in different formats, for example:
  - Tait Bryan Angles (Roll, Pitch, Yaw)
  - Quaternions

Attitude Determination – Gyroscope

Gyroscope measures the angular velocity in 3 axis

$$\omega = \left[\omega_x \; \omega_y \; \omega_z\right]^T = \left[\dot{\phi} \;\; \dot{\theta} \;\; \dot{\psi}\right]^T$$

→ Using numerical Integration with an initial State:

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}_t = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}_{t-1} + \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}_t \cdot \Delta t$$

Attitude Determination – Gyroscope

Gyroscope measures the angular velocity in 3 axis

$$\omega = \left[\omega_x \; \omega_y \; \omega_z\right]^T = \left[\dot{\phi} \;\; \dot{\theta} \;\; \dot{\psi}\right]^T \qquad \Omega = \left[0 \;\; \omega_x \; \omega_y \; \omega_z\right]^T$$

→ Update the quaternion with the gyroscope

$$q_{\mathrm{w,t}} = q_{t-1} + \dot{q}_{w,t} \qquad \textbf{with} \qquad \dot{q}_{w,t} = \frac{\Delta t}{2}\left(\Omega_t \otimes q_{t-1}\right)$$

Attitude Determination – Gyroscope

Gyroscope measures the angular velocity in 3 axis

$$\omega = \left[\omega_x \; \omega_y \; \omega_z\right]^T = \left[\dot{\phi} \;\; \dot{\theta} \;\; \dot{\psi}\right]^T \qquad \Omega = \left[0 \;\; \omega_x \; \omega_y \; \omega_z\right]^T$$

→ Update the quaternion with the gyroscope

$$q_{\mathrm{w,t}} = q_{t-1} + \frac{\Delta t}{2}\left(\Omega_t \otimes q_{t-1}\right) = \begin{bmatrix} 1 & -\frac{\Delta t}{2}\omega_x & -\frac{\Delta t}{2}\omega_y & -\frac{\Delta t}{2}\omega_z \\ \frac{\Delta t}{2}\omega_x & 1 & \frac{\Delta t}{2}\omega_z & -\frac{\Delta t}{2}\omega_y \\ \frac{\Delta t}{2}\omega_y & -\frac{\Delta t}{2}\omega_z & 1 & \frac{\Delta t}{2}\omega_x \\ \frac{\Delta t}{2}\omega_z & \frac{\Delta t}{2}\omega_y & -\frac{\Delta t}{2}\omega_x & 1 \end{bmatrix} \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} q_w - \frac{\Delta t}{2}\omega_x q_x - \frac{\Delta t}{2}\omega_y q_y - \frac{\Delta t}{2}\omega_z q_z \\ q_x + \frac{\Delta t}{2}\omega_x q_w - \frac{\Delta t}{2}\omega_y q_z + \frac{\Delta t}{2}\omega_z q_y \\ q_y + \frac{\Delta t}{2}\omega_x q_z + \frac{\Delta t}{2}\omega_y q_w - \frac{\Delta t}{2}\omega_z q_x \\ q_z - \frac{\Delta t}{2}\omega_x q_y + \frac{\Delta t}{2}\omega_y q_x + \frac{\Delta t}{2}\omega_z q_w \end{bmatrix}$$

Attitude Determination - Accelerometer

Accelerometer measures the linear acceleration in 3 axis

$$a = [a_x \; a_y \; a_z]^T$$

→ Using the gravity vector to calculate the attitude

$$\phi_{acc} = atan2\left(-a_x \, , \sqrt{a_y^2 + a_z^2}\right) \qquad \theta_{acc} = atan2(a_y, a_z)$$

Attitude Determination – Magnetometer

Rotate/Compensate the measurement by the current tilt

$$\widetilde{m} = \begin{bmatrix} \cos\theta & \sin\theta\sin\phi & \sin\theta\cos\phi \\ 0 & \cos\phi & -\sin\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{bmatrix} \cdot \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix} = \begin{bmatrix} m_x\cos\theta + m_y\sin\theta\sin\phi + m_z\sin\theta\cos\phi \\ m_y\cos\phi - m_z\sin\phi \\ -m_x\sin\theta + m_y\cos\theta\sin\phi + m_z\cos\theta\cos\phi \end{bmatrix}$$

Calculate Yaw:

$$\psi = atan2\left(-\widetilde{m_y}, \widetilde{m_x}\right)$$

Data combination

- **Low- and High-Pass filters are combined when the data of the sensors is added**
- **Data combination for the complimentary filter:**

$$\begin{bmatrix}\phi\\ \theta\\ \psi\end{bmatrix}_t = \underbrace{(1-\beta)\cdot\left(\begin{bmatrix}\phi\\ \theta\\ \psi\end{bmatrix}_{t-1} + \begin{bmatrix}\phi_{gyro}\\ \theta_{gyro}\\ \psi_{gyro}\end{bmatrix}_t\right)}_{\text{High-Pass Filter for Gyroscope Angles}} + \underbrace{\beta\cdot\begin{bmatrix}\phi_{acc}\\ \theta_{acc}\\ \psi_{mag}\end{bmatrix}_t}_{\text{Low-Pass Filter for Acc/Mag Angles}}$$

Data combination with Quaternion

- **Transfer the pitch and roll angle measurements from the accelerometer and the yaw from the magnetometer to a quaternion:**

$$\mathbf{q}_{am} = \begin{pmatrix} q_w \\ q_x \\ q_y \\ q_z \end{pmatrix} = \begin{pmatrix} \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) - \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \\ \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) - \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) \end{pmatrix}$$

Data combination with Quaternion

- **Final combination for the complimentary filter:**

$$\mathbf{q_t} = (1 - \beta) \cdot q_{w,t} + \beta \cdot q_{am,t}$$

- **To use the quaternion for the attitude we need to normalize it!**

$$\hat{\boldsymbol{q}} = \frac{\mathbf{q}}{||\boldsymbol{q}||}$$

Task2.2: Simple Quaternion Filter

Implement a simple Quaternion filter

- Use the formulas given in this document
- Implement the filter (Look at the very similar previous task 1)
- Compare the two filters (complementary vs quaternion) in realtime

Part 2: Questions

Please answer the following questions:

- **Name one real world problem for each of the sensor in the IMU**
- **What are quaternions and why do we need them?**
- **What is the matlib?**

Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Chair of Aerospace Information Technology

Online Floatsat 2021
Attitude Determination – Part 2

Julian Rothe